test

RoLE model set-up

```
library(roleR)
library(ape)

p <- roleParams(alpha = 10000, dispersal_prob = 0.1,
                niter = 1000000, niterTimestep = 500000)
m <- roleModel(p)
r <- runRole(m)

finalPhylo <- r@modelSteps[[length(r@modelSteps)]]@phylo

nspp <- finalPhylo@n

tBySpp <- sapply(r@modelSteps, function(d) {
    x <- tabulate(d@localComm@indSpecies)
    n <- length(x)
    x <- c(x, rep(0, nspp - n))
}) |> t()


# write stuff out ----
f <- "passingR2Py"

# localAbund (but not spp indeces, actual abunds through time),
# for now let `spAbundHarmMean` by equal to abund
# time info added
cbind(r@info[, c("timestep", "generations")], tBySpp) |>
    write.csv(file = paste(f, "time_sp-local.csv", sep = "/"),
              row.names = FALSE)
```

```
# metaAbund
round(r@modelSteps[[1]]@metaComm@spAbund * r@info$individuals_meta[1]) |>
    matrix() |>
    write.csv(file = paste(f, "metaAbund.csv", sep = "/"),
              row.names = FALSE, col.names = FALSE)
```

```
Warning in write.csv(matrix(round(r@modelSteps[[1]]@metaComm@spAbund *
r@info$individuals_meta[1])), : attempt to set 'col.names' ignored
```

```
# localTDiv, need to make something arbitrary for now

# metaTree
as(finalPhylo, "phylo") |>
    write.tree(file = paste(f, "phy.nwk", sep = "/"))
```

You can add options to executable code like this

```
import msprime
import newick
import numpy as np
import pandas as pd
from collections import Counter, defaultdict
from typing import List, Dict, Tuple, Optional, Union


def _force_ultrametric(tree):
    """
    Force all leaf nodes to time 0
    Input a newick tree in string format
    Output is a modified newick tree with all leaves having equal total length
    """
    # Parse the newick tree string.
    parsed = newick.loads(tree)
    if len(parsed) == 0:
        raise ValueError(f"Not a valid newick tree: '{tree}'")
    root = parsed[0]

    # Set node depths (distances from root).
    stack = [(root, 0)]
    max_depth = 0
    while len(stack) > 0:
```

```
        node, depth = stack.pop()
        if depth > max_depth:
            max_depth = depth
        node.depth = depth
        for child in node.descendants:
            stack.append((child, depth + child.length))
    for node in root.walk():
        if node.is_leaf:
            ## Add offset to node.length to foce all nodes to fall at time 0
            ## The offset is the difference between the depth of this node
            ## and the max_depth of the deepest leaf node.
            node.length = node.length + (max_depth - node.depth)
    return root.newick

# read in things from R
time_sp = pd.read_csv("passingR2Py/time_sp-local.csv")
meta_abun = pd.read_csv("passingR2Py/metaAbund.csv")

with open("passingR2Py/phy.nwk", "r") as file:
    nwk = file.read()

phylo_meta = _force_ultrametric(nwk)

phylo_meta
```

```
'(t2:4.983258130700001,(t10:4.294589995100002,((t1:0.8648863060000009,((t3:0.296435883700000
```